নিচে **Django Query / ORM Interview Questions** গুলো **basic → advanced** লেভেলে সাজিয়ে দিলাম। এগুলো খুবই কমন এবং practical 🔥
(তুমি যেহেতু Django practice করছো—এগুলো especially important)

🔹 Basic Django ORM Interview Questions

1. Django ORM কী?

Answer:
Django ORM (Object Relational Mapping) এমন একটি system যা Python class দিয়ে database table represent করে এবং SQL না লিখেই query করা যায়।

---

2. `Model.objects.all()` কী করে?

Answer:
ডাটাবেস থেকে model-এর সব record এনে দেয় (QuerySet return করে)।

---

3. `filter()` এবং `get()` এর পার্থক্য কী?

Answer:

filter()	get()
একাধিক result দিতে পারে	শুধু একটাই object
QuerySet return করে	single object
error দেয় না	Multiple/DoesNotExist error দেয়

```
User.objects.filter(is_active=True)
User.objects.get(id=1)
```

---

4. `first()` vs `last()` কী?

```
User.objects.first()
User.objects.last()
```

👉 প্রথম বা শেষ object return করে, না থাকলে `None`

---

5. `exists()` কেন ব্যবহার করা হয়?

Answer:
Data আছে কিনা check করার জন্য (efficient)

```
User.objects.filter(email=email).exists()
```

🔹 Intermediate Django Query Questions

6. `exclude()` কী করে?

Answer:
Condition match করা data বাদ দেয়

```
User.objects.exclude(is_staff=True)
```

---

7. `values()` এবং `values_list()` এর পার্থক্য?

```
User.objects.values('id', 'username')
User.objects.values_list('id', 'username')
```

values	values_list
dict return করে	tuple return করে

---

8. `Q object` কেন দরকার?

Answer:
Complex query (OR, AND, NOT) করার জন্য

```
from django.db.models import Q

User.objects.filter(Q(is_staff=True) | Q(is_superuser=True))
```

9. `F object` কী?

Answer:
Database field এর সাথে field operation করার জন্য

```
from django.db.models import F

Product.objects.update(stock=F('stock') - 1)
```

---

10. `order_by()` কী করে?

```
User.objects.order_by('-date_joined')
```

👉 descending order (`-` মানে DESC)

🔹 Advanced Django ORM Interview Questions

11. `select_related()` vs `prefetch_related()`?

select_related	prefetch_related
SQL JOIN ব্যবহার করে	আলাদা query
ForeignKey, OneToOne	ManyToMany

```
Book.objects.select_related('author')
Book.objects.prefetch_related('categories')
```

---

12. N+1 Query Problem কী?

Answer:
একটা main query এর সাথে প্রতিটা object এর জন্য extra query হওয়া → performance slow হয়
Solution 👉 `select_related`, `prefetch_related`

---

13. `annotate()` vs `aggregate()`?

```
from django.db.models import Count

Book.objects.annotate(total=Count('reviews'))
Book.objects.aggregate(total=Count('id'))
```

annotate	aggregate
per object result	single result

---

14. `distinct()` কেন ব্যবহার হয়?

Answer:
Duplicate row remove করতে

```
User.objects.values('email').distinct()
```

---

15. `update()` vs `save()`?

update()	save()
database direct	model instance
signals trigger হয় না	signals trigger হয়

🔹 Tricky / Frequently Asked

16. `QuerySet` lazy কেন?

Answer:
Query তখনই database এ যায় যখন data actually দরকার হয় (performance benefit)

---

17. `bulk_create()` কখন ব্যবহার করবে?

Answer:
Multiple objects insert করার সময় (fast)

---

18. `raw()` query কখন লাগতে পারে?

Answer:
Complex SQL বা DB-specific feature দরকার হলে

---

19. `only()` এবং `defer()` ?

```
User.objects.only('username')
User.objects.defer('password')
```

---

20. Django ORM vs Raw SQL?

Answer:
ORM safer, readable, portable
Raw SQL faster but risky

🔹 Bonus: Real Interview Coding Question

👉 **Active users যাদের last_login 30 দিনের মধ্যে**

```
from django.utils.timezone import now
from datetime import timedelta

User.objects.filter(last_login__gte=now() - timedelta(days=30))
```

তুমি চাইলে আমি:

✅ MCQ version
✅ Answer short version
✅ Real project-based query
✅ DRF Query interview Q&A

বললেই হবে 👍

চমৎকার 👍
নিচে **Django REST Framework (DRF) Query Interview Q&A** দেওয়া হলো—**most asked**,

practical, এবং **interview-ready** format এ।
(Beginner → Advanced)

🔹 DRF Basic Query Interview Questions

1. DRF-এ queryset কোথায় define করা হয়?

Answer:
`APIView`, `GenericAPIView`, বা `ViewSet`-এ `queryset` attribute হিসেবে।

```
class UserViewSet(ModelViewSet):
    queryset = User.objects.all()
```

---

2. `get_queryset()` কেন ব্যবহার করা হয়?

Answer:
Dynamic / request-based queryset বানানোর জন্য।

```
def get_queryset(self):
    return User.objects.filter(is_active=True)
```

---

3. `query_params` কী?

Answer:
Request এর URL parameter access করার জন্য।

```
self.request.query_params.get('search')
```

---

4. `request.GET` আর `query_params` পার্থক্য?

Answer:
DRF-এ `query_params` preferred (framework-agnostic)

🔹 Filtering Interview Questions

5. DRF-এ filtering করার কয়টা উপায়?

Answer:
1️⃣ Manual filtering (`get_queryset`)
2️⃣ `django-filter`
3️⃣ Custom filter backend

---

6. Manual filtering example

```
def get_queryset(self):
    qs = Product.objects.all()
    category = self.request.query_params.get('category')
    if category:
        qs = qs.filter(category__name=category)
    return qs
```

---

7. `django-filter` কী?

Answer:
Query parameter দিয়ে automatic filtering করার library।

```
filter_backends = [DjangoFilterBackend]
filterset_fields = ['status', 'price']
```

---

8. `filterset_fields` vs `FilterSet`?

filterset_fields	FilterSet
simple	advanced
limited	custom logic

🔹 Search & Ordering Questions

9. `SearchFilter` কী?

Answer:
Search query parameter (`?search=`) দিয়ে search।

```
filter_backends = [SearchFilter]
search_fields = ['name', 'description']
```

---

10. Partial search কীভাবে হয়?

Answer:
`icontains` internally ব্যবহার হয়।

---

11. `OrderingFilter` কী?

```
filter_backends = [OrderingFilter]
ordering_fields = ['price', 'created_at']
```

URL:

```
?ordering=-price
```

🔹 Pagination Interview Questions

12. Pagination কেন দরকার?

Answer:
Large dataset এ performance + memory optimize করার জন্য।

---

13. DRF Pagination কয় প্রকার?

Answer:
1️⃣ PageNumberPagination
2️⃣ LimitOffsetPagination
3️⃣ CursorPagination

---

14. PageNumberPagination example

```
?page=2
```

---

15. CursorPagination কখন ব্যবহার করবে?

Answer:
Large & frequently updated dataset এ (stable ordering)

🔹 Advanced DRF Query Questions

16. `select_related()` DRF-এ কোথায় ব্যবহার হয়?

Answer:
`queryset` বা `get_queryset()` এ।

```
queryset = Order.objects.select_related('user')
```

---

17. N+1 problem DRF-এ কীভাবে হয়?

Answer:
Serializer nested field এর জন্য extra queries।

✔ Solution:

```
queryset = Order.objects.select_related('user').prefetch_related('items')
```

---

18. Serializer query optimization কিভাবে করবে?

Answer:

- `select_related`
- `prefetch_related`
- `SerializerMethodField` এ query avoid

---

19. `annotate()` DRF-এ কিভাবে ব্যবহার হয়?

```
Product.objects.annotate(total_reviews=Count('reviews'))
```

Serializer:

```
total_reviews = serializers.IntegerField(read_only=True)
```

---

20. `distinct()` কখন দরকার?

Answer:
ManyToMany বা join এ duplicate row আসলে।

🔹 Security & Performance

21. DRF-এ data expose কমাতে কী করবে?

Answer:

- Serializer field control
- Permission class
- Queryset filtering by user

```
def get_queryset(self):
    return Order.objects.filter(user=self.request.user)
```

---

22. Heavy query slow হলে কী করবে?

Answer:

- Index add
- Pagination
- Cache
- ORM optimization

🔹 Tricky Interview Questions

23. `update()` vs `serializer.save()`?

update()	serializer.save()
bulk fast	validation + signal
signal নেই	signal থাকে

24. DRF-এ raw SQL ব্যবহার করা যায়?

Answer:
হ্যাঁ, কিন্তু discouraged।

---

25. Real Interview Scenario Question

Q:
“User যেন শুধু নিজের data দেখতে পারে—কিভাবে?”

A:

```
def get_queryset(self):
    return Post.objects.filter(author=self.request.user)
```

🔥 One-Line Rapid Revision (Interview Favorite)

- `query_params` → URL filtering
- `get_queryset()` → dynamic data
- `django-filter` → clean filtering
- `SearchFilter` → text search
- `OrderingFilter` → sorting
- `select_related` → FK
- `prefetch_related` → M2M

চাও তো আমি:
✅ **MCQ format**
✅ **30 rapid-fire Q&A**
✅ **Real company interview scenario**
✅ **DRF + SQL optimization questions**

বললেই হবে 💪

নিশ্চিত 👍
নিচে **DRF Query – 30 Rapid-Fire Interview Q&A** দেওয়া হলো।
👉 **সব প্রশ্ন আলাদা**, short & crisp (1–2 line answer) — interview-friendly 🔥

🔥 DRF Query – 30 Rapid-Fire Q&A

1. DRF-এ queryset lazy কেন?

Ans: Database hit তখনই হয় যখন data evaluate হয়।

---

2. `get_queryset()` কখন call হয়?

Ans: প্রতিটি request এর সময়।

---

3. `queryset` attribute vs `get_queryset()` ?

Ans: Static vs dynamic queryset।

---

4. DRF-এ default pagination off থাকে?

Ans: হ্যাঁ, manually enable করতে হয়।

---

5. `query_params` immutable কেন?

Ans: Request data integrity রক্ষার জন্য।

---

6. `request.data` আর `query_params` difference?

Ans: data → body, query_params → URL।

---

7. DRF-এ filter backend কোথায় define হয়?

Ans: View বা global settings এ।

---

8. `django-filter` না থাকলে কী হবে?

Ans: Filter backend error দিবে।

9. Multiple filter backend একসাথে কাজ করে?

Ans: হ্যাঁ, sequentially।

---

10. `SearchFilter` case-sensitive?

Ans: না, case-insensitive।

---

11. Search multiple fields possible?

Ans: হ্যাঁ, list আকারে।

---

12. Ordering default order কিভাবে set করবে?

Ans: `ordering = ['-created_at']`

---

13. Pagination response এ কী extra আসে?

Ans: count, next, previous, results।

---

14. PageNumberPagination limit hard?

Ans: না, configurable।

---

15. CursorPagination কেন fast?

Ans: OFFSET এড়ায়।

---

16. Serializer এ queryset define করা উচিত?

Ans: না, view layer responsibility।

---

17. Nested serializer slow কেন?

Ans: Extra DB query তৈরি করে।

---

18. `select_related()` কখন কাজ করবে না?

Ans: ManyToMany relation এ।

---

19. `prefetch_related()` কিভাবে কাজ করে?

Ans: Python-level join।

---

20. DRF-এ duplicate data আসার কারণ?

Ans: Join + missing `distinct()` ।

---

21. Large data serialize slow কেন?

Ans: Heavy field + no pagination।

---

22. `SerializerMethodField` expensive কেন?

Ans: প্রতি object এ call হয়।

---

23. Filtering permission layer এ করা যায়?

Ans: হ্যাঁ, কিন্তু recommended না।

---

24. Query optimization responsibility কার?

Ans: View / queryset layer।

---

25. DRF-এ DB hit detect কিভাবে?

Ans: Django Debug Toolbar।

---

26. `count()` vs `len(queryset)`?

Ans: count() → DB level, len() → evaluate all।

---

27. DRF-এ custom query param নাম দিতে পারি?

Ans: হ্যাঁ।

---

28. ViewSet এ filtering disable কিভাবে?

Ans: filter_backends remove করে।

---

29. DRF-এ performance bottleneck সাধারণত কোথায়?

Ans: Serializer + DB queries।

---

30. Interview Golden Rule (1 line)

Ans:
👉 *Optimize queryset first, serializer later.*

💡 Interview Tip

যদি interviewer জিজ্ঞেস করে:

> "DRF slow কেন?"

Perfect Answer:
"Unoptimized queryset, missing pagination, and nested serializer queries."

চাও তো আমি পরের ধাপে দিতেই পারি:
✅ **DRF System Design Q&A**
✅ **DRF Performance Debug Case**
✅ **Company-wise DRF interview pattern (BJIT, Brain Station, Enosis)**

বললেই হবে 🔥

Complete Django & DRF Query Guide (Apply to Any Project)

This guide is a **practical, end-to-end reference** you can apply to **any Django / DRF project**—from CRUD apps to large production APIs. Use it as a checklist while building features.

---

1) Core Concepts (Must Know)

QuerySet

- Lazy, chainable, reusable
- Hits DB only when evaluated (iterate, list(), len(), bool())

```
qs = User.objects.filter(is_active=True)
```

Evaluation Triggers

- for loop, list(qs)
- qs[0]
- len(qs)
- bool(qs)

---

2) CRUD Queries (Daily Use)

Create

```
User.objects.create(username='john')
```

Read

```
User.objects.all()
User.objects.get(id=1)
User.objects.filter(is_active=True)
```

Update

```
User.objects.filter(id=1).update(is_active=False)
user.is_active = False
user.save()
```

Delete

```
User.objects.filter(id=1).delete()
```

---

3) Filtering (Most Used)

Field Lookups

```
price__gte=100
name__icontains='phone'
created_at__date=date.today()
```

exclude()

```
User.objects.exclude(is_staff=True)
```

Q Objects (OR / NOT)

```
from django.db.models import Q
User.objects.filter(Q(is_staff=True) | Q(is_superuser=True))
```

---

4) Sorting & Limiting

```
User.objects.order_by('-created_at')
User.objects.all()[:10]
```

---

5) Selecting Fields (Optimization)

values / values_list

```
User.objects.values('id','username')
User.objects.values_list('id','username', flat=False)
```

only / defer

```
User.objects.only('username')
User.objects.defer('password')
```

---

6) Relationships (CRITICAL)

ForeignKey / OneToOne → select_related

```
Order.objects.select_related('user')
```

ManyToMany / Reverse FK → prefetch_related

```
Order.objects.prefetch_related('items')
```

N+1 Fix Pattern

```
queryset = Post.objects.select_related('author').prefetch_related('tags')
```

---

7) Aggregation & Annotation

aggregate (single result)

```
from django.db.models import Avg
Product.objects.aggregate(avg_price=Avg('price'))
```

annotate (per row)

```
from django.db.models import Count
Product.objects.annotate(review_count=Count('reviews'))
```

8) Advanced Expressions

F Expressions

```
from django.db.models import F
Product.objects.update(stock=F('stock')-1)
```

Case / When

```
from django.db.models import Case, When, Value, IntegerField
User.objects.annotate(
    role=Case(
        When(is_superuser=True, then=Value(1)),
        default=Value(0),
        output_field=IntegerField()
    )
)
```

---

9) Performance Tools

count vs len

```
qs.count()   # DB level
len(qs)      # loads all rows
```

exists

```
User.objects.filter(email=email).exists()
```

bulk operations

```
User.objects.bulk_create(users)
```

---

10) DRF Query Integration

Base ViewSet Pattern

```
class ProductViewSet(ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
```

Dynamic Queryset

```
def get_queryset(self):
    qs = Product.objects.all()
    category = self.request.query_params.get('category')
    if category:
        qs = qs.filter(category__name=category)
    return qs
```

---

11) DRF Filtering / Search / Ordering

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import SearchFilter, OrderingFilter

filter_backends = [DjangoFilterBackend, SearchFilter, OrderingFilter]
filterset_fields = ['status','price']
search_fields = ['name','description']
ordering_fields = ['price','created_at']
```

---

12) Pagination (Always Use)

```
from rest_framework.pagination import PageNumberPagination

class DefaultPagination(PageNumberPagination):
    page_size = 10
```

---

13) Security-Aware Querying

User-Scoped Data

```
def get_queryset(self):
    return Order.objects.filter(user=self.request.user)
```

---

14) Common Project Patterns

Soft Delete

```
objects.filter(is_deleted=False)
```

Date Range Filter

```
created_at__range=[start,end]
```

Search + Filter Combined

```
qs.filter(name__icontains=search).filter(status=status)
```

---

15) When to Use Raw SQL

- Complex reporting
- DB-specific feature
- Extreme performance need

```
Model.objects.raw("SELECT * FROM table")
```

---

16) Debug & Optimize

- Django Debug Toolbar
- print(connection.queries)
- add DB indexes

---

17) Golden Rules (Interview + Production)

1. Filter early
2. Paginate always
3. select_related / prefetch_related
4. Avoid SerializerMethodField queries
5. count() > len()

---

18) Universal Checklist (Any Project)

- [ ] Pagination enabled
- [ ] Query optimized
- [ ] User-scoped data
- [ ] Index on heavy filters
- [ ] No N+1 issue

You can apply this guide to:

- E-commerce
- LMS
- Scholarship system
- Blog / Social app
- Admin dashboards

If you want next:

- Convert this into **PDF / cheat-sheet**
- **Project-specific query design** (e-commerce / scholarship)
- **Interview mock test**